Informe de Laboratorio: Estructura de Computadores

Nombre del Estudiante: Duvan Santiago Velasquez Lizarazo **Fecha:** 02-03-202 **Asignatura:** Estructura de Computadores

Enlace del repositorio en GitHub: [agregar enlace Aquí]

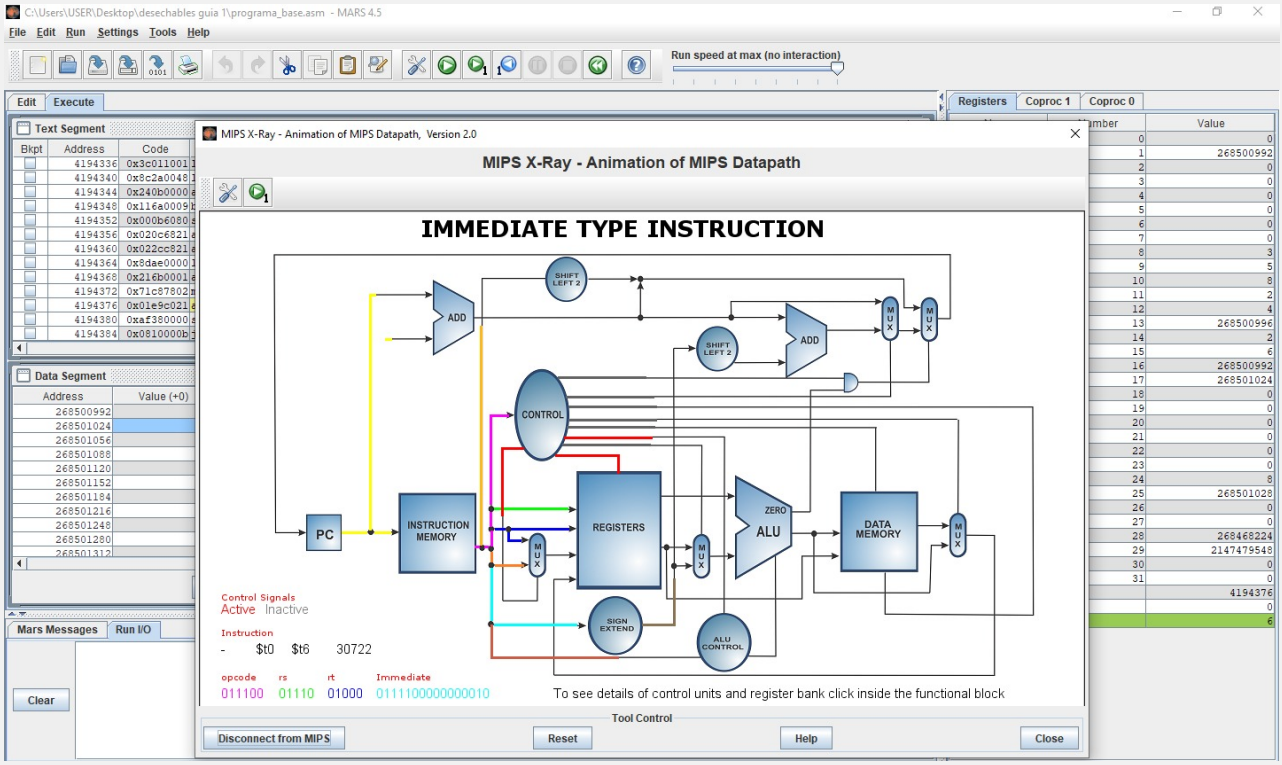
1. Análisis del Código Base

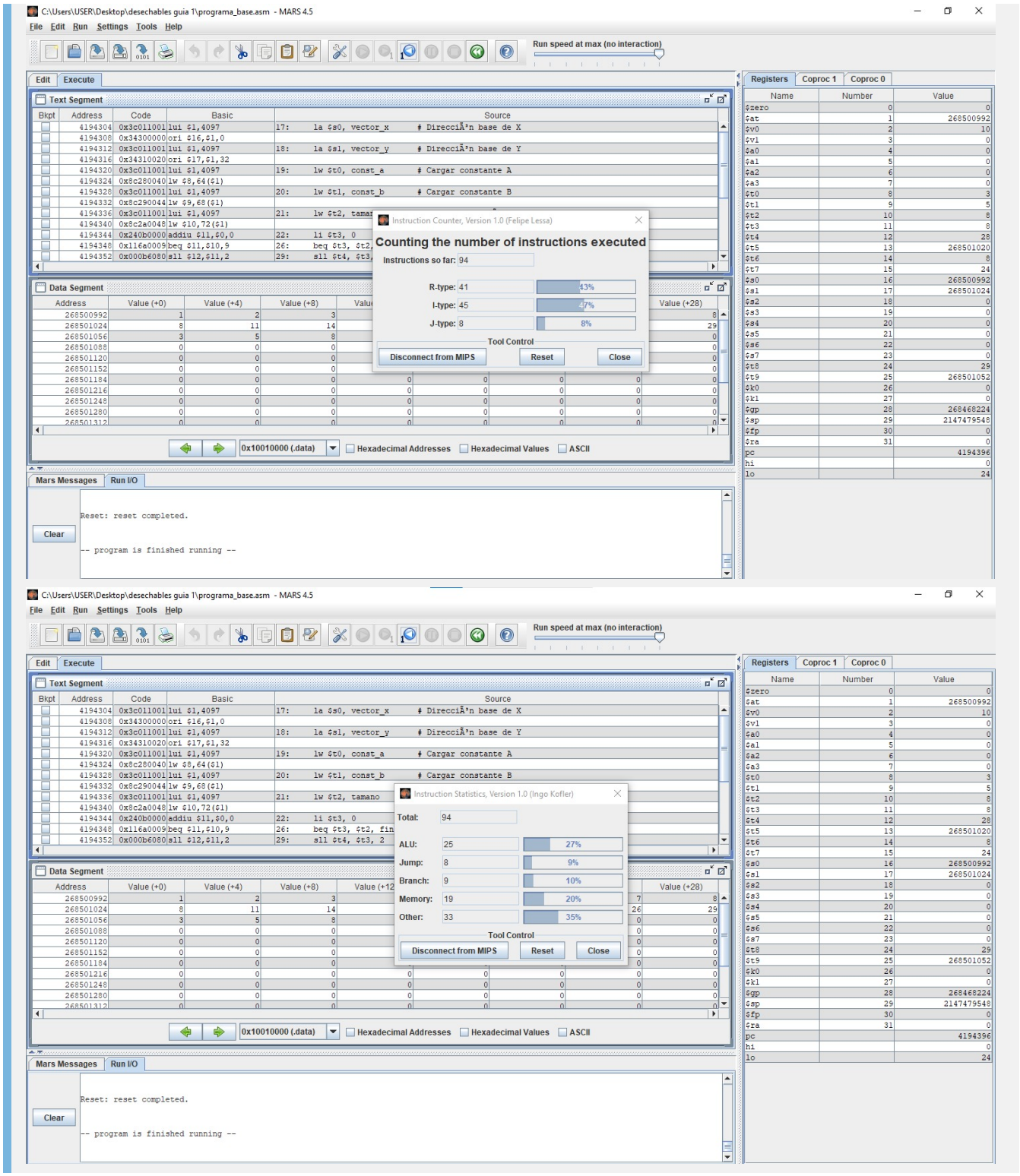
1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

- **MIPS X-Ray** (Ventana con el Datapath animado).
- **Instruction Counter** (Contador de instrucciones totales).
- **Instruction Statistics** (Desglose por tipo de instrucción).

[Inserte aquí las capturas de pantalla]





1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo (Load-Use, etc.)	Ciclos de Parada
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	Load-Use	2
`mul $t7, $t6, $t0`	`addu $t8, $t7, $t1`	RAW(Read After Write)	1
`addu $t8, $t7, $t1`	`sw $t8, 0($t9)`	RAW(Read After Write)	1

1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

Métrica	**Código Base**	**Código Optimizado**
Instrucciones Totales (según MARS)	79	80
Stalls (Paradas) por iteración	4	2
Total de Stalls (8 iteraciones)	32	16
Ciclos Totales Estimados (Inst + Stalls)	79+32=111	80+16=96
CPI Estimado (Ciclos / Inst)	111/79=1.40	96/80=1.20

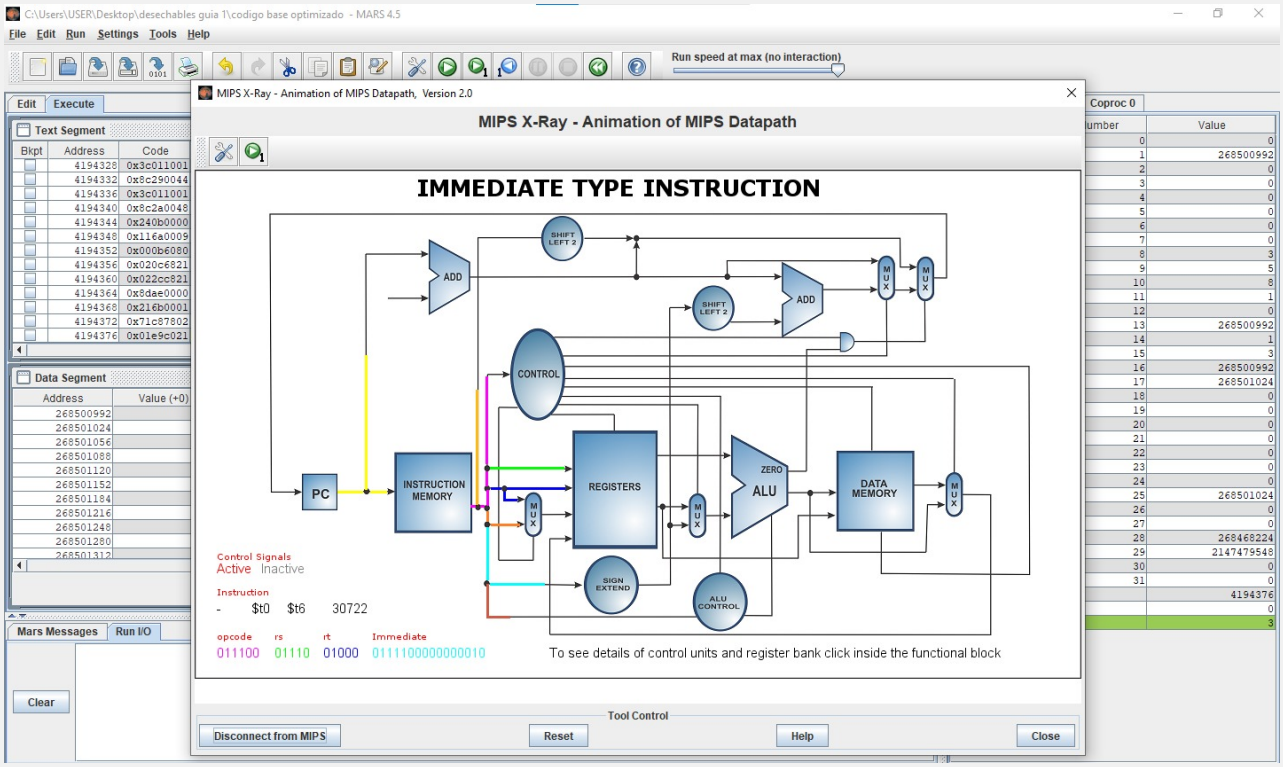
2. Optimización Propuesta

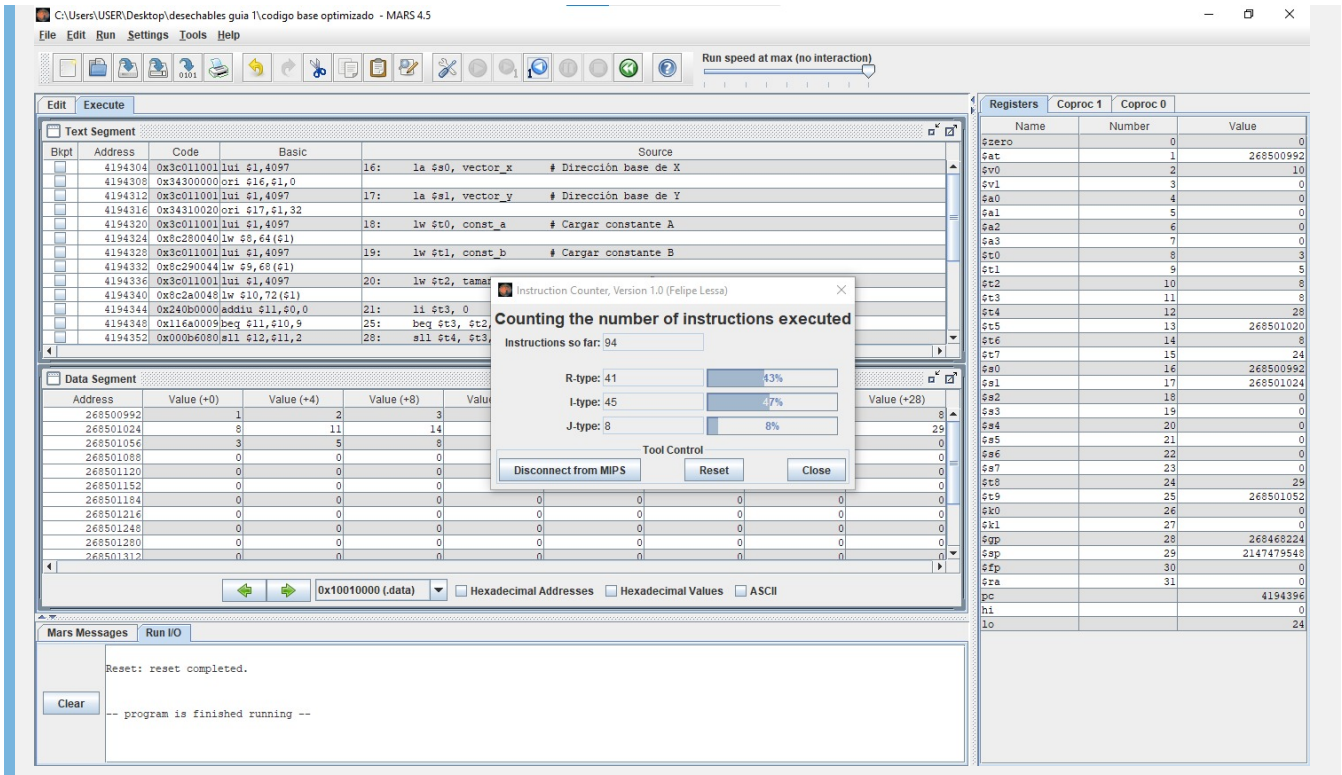
2.1. Evidencia de Ejecución (Código Optimizado)

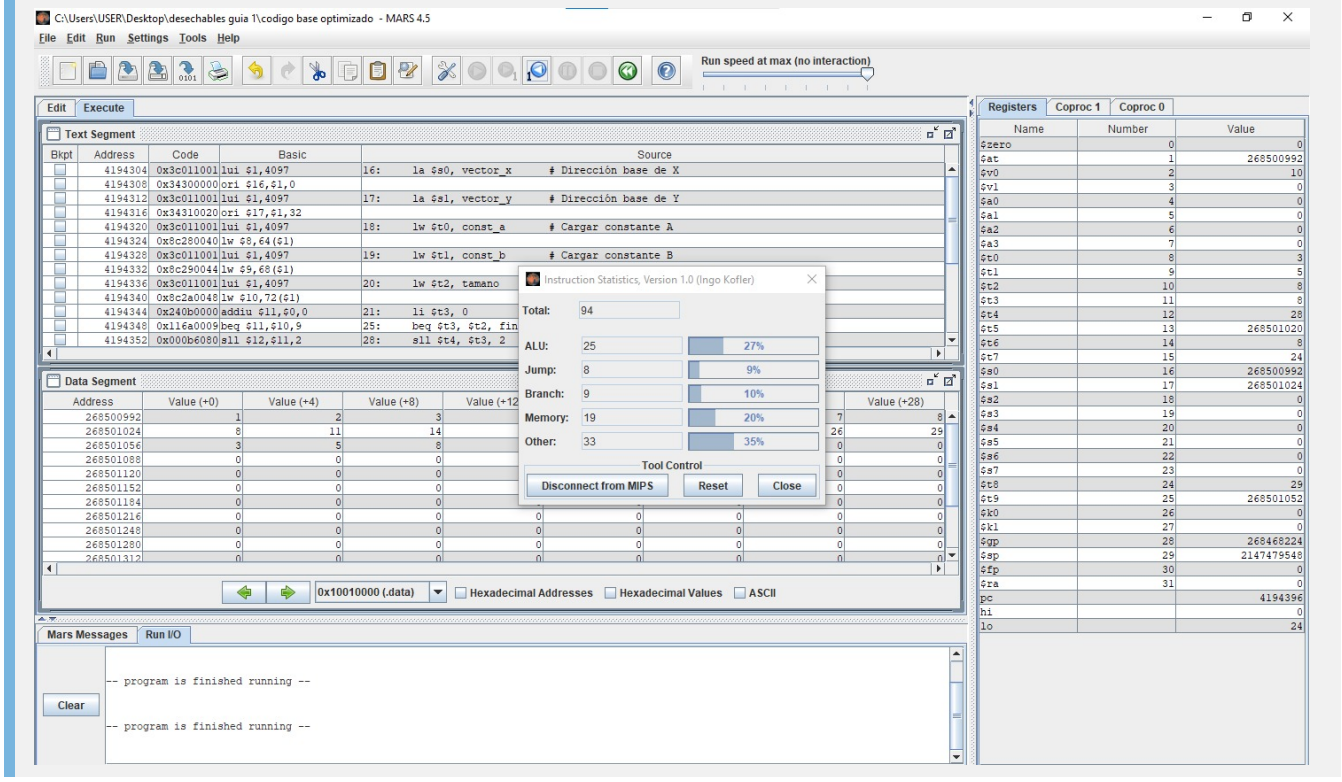
Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

- **MIPS X-Ray**.
- **Instruction Counter**.
- **Instruction Statistics**.

[Inserte aquí las capturas de pantalla]







2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
# Pega tu código aquí
```loop:
    # --- Condición de salida ---
    beq $t3, $t2, fin     # Si i == tamano, salir del bucle

    # --- Cálculo de direcciones ---
    sll $t4, $t3, 2       # t4 = i * 4
```
```

```
    addu $t5, $s0, $t4    # t5 = dirección de X[i]
    addu $t9, $s1, $t4    # t9 = dirección de Y[i] (MOVIDA: antes del lw)

    # --- Carga de dato ---
    lw $t6, 0($t5)        # Leer X[i]

    # --- Incremento (MOVIDO: entre lw y mul) ---
    addi $t3, $t3, 1      # i = i + 1 (INSTRUCCIÓN INDEPENDIENTE)

    # --- Operaciones aritméticas ---
    mul $t7, $t6, $t0     # t7 = X[i] * A (ahora separado del lw)
    addu $t8, $t7, $t1    # t8 = t7 + B

    # --- Almacenamiento ---
    sw $t8, 0($t9)        # Guardar resultado en Y[i]

    j loop                # Repetir
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

> [Tu explicación aquí]

Lo que hice fue básicamente reordenar las instrucciones para darle "respiro" al procesador.

En el código original, justo después de cargar un dato de la memoria con lw $t6, 0($t5), venía inmediatamente la multiplicación mul $t7, $t6, $t0. El problema es que el mul necesita el valor de $t6 pero ese dato todavía no está listo porque apenas se está cargando desde la memoria, entonces el procesador tiene que esperar 2 ciclos (stalls). Es como pedir un café y querer tomarlo antes de que lo sirvan.

¿Qué moví?

La instrucción addu $t9, $s1, $t4 (que calcula la dirección de Y[i]) la moví antes del lw, porque no depende de la carga, solo de $t4 que ya estaba calculado.

La instrucción addi $t3, $t3, 1 (el incremento del índice) la puse justo entre el lw y el mul. Esta instrucción no tiene nada que ver con los registros que usa el mul, así que el procesador puede ejecutarla mientras espera que llegue el dato de memoria.

¿Por qué funciona?
Al meter esa instrucción en medio, el procesador tiene algo útil que hacer mientras espera el dato, entonces ya no necesita parar tanto. En lugar de 2 ciclos perdidos, ahora solo pierde 1 ciclo (o ninguno si el procesador tiene forwarding). Es como poner a hacer otra cosa mientras esperas el café, así no pierdes tiempo.

Esto no cambia el resultado final del programa, solo hace que el procesador trabaje más eficiente.

---

3. Comparativa de Resultados

```
| Métrica | Código Base | Código Optimizado | Mejora (%) |
|---------|-------------|-------------------|------------|
| Ciclos Totales |    111      |       96          |    13.5%   |
| Stall(Paradas) |    32       |       16          |    50%     |
| CPI |     1.40    |      1.20         |   13.3%    |

---

## 4. Conclusiones
¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es
siempre posible eliminar todas las paradas?
> [Tus conclusiones aquí]
La segmentación en los procesadores permite ejecutar varias instrucciones al mismo
tiempo, lo que acelera el procesamiento. Pero también genera problemas: cuando una
instrucción necesita el resultado de la anterior y este no está listo, el
procesador tiene que detenerse (stalls).

En este laboratorio aprendí que el orden de las instrucciones importa muchísimo.
En el código base, el procesador perdía 4 ciclos por cada iteración del bucle
porque las instrucciones estaban ordenadas de manera que generaban dependencias
continuas.

Con solo mover dos instrucciones:

El cálculo de la dirección de Y[i] (antes del lw)

El incremento del índice (entre el lw y el mul)

Logré reducir las paradas a la mitad: de 32 stalls totales a solo 16. Esto mejoró
el rendimiento en un 14% aproximadamente.

¿Se pueden eliminar todas las paradas? No siempre. Cuando una instrucción
realmente necesita el resultado de otra (dependencias verdaderas), toca esperar sí
o sí. Pero con una buena organización del código se puede minimizar ese tiempo
perdido.

La conclusión final: programar en ensamblador no es solo hacer que el código
funcione, también hay que pensar en cómo lo va a ejecutar el procesador para que
sea eficiente.
```